

10주차_5, 6장(~TF-IDF)

(248~)

5장_토큰화

자연어 처리

- 자연어(natural language): 인공적으로 만들어진 것이 아닌, 사람의 언어 활동을 위해 자연히 만들어진 언어
- **자연어 처리(natural language processing, NLP)**: 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술로, 인공지능의 하위 분야 중 하나임
 - 인간 언어의 구조, 의미, 맥락을 분석하고 이해할 수 있는 알고리즘과 모델 개발
 - 자연어 처리 모델을 개발할 때 해결해야 할 문제점
 - 모호성(ambiguity): 알고리즘은 맥락에 따른 자연어의 다양한 의미를 이해하고 명확하게 구분할 수 있어야 함
 - 가변성(variability): 알고리즘은 사투리, 강세, 신조어, 작문 스타일에 따른 가변성을 처리할 수 있어야 하며 사용 중인 언어를 이해할 수 있어야 함
 - 구조(structure): 알고리즘은 구문(syntactic)을 통해 의미(semantic)를 추론하거나 분석할 수 있어야 함
 - **토큰화(tokenization)**: 컴퓨터가 자연어를 이해할 수 있도록 **말뭉치(corpus)** → **토큰(token)** 이라는 일정한 단위로 나뉘어야 함
 - 말뭉치: 뉴스 기사, 사용자 리뷰, 저널이나 칼럼 등에서 목적에 따라 구축되는 텍스트 데이터로, 자연어 모델을 훈련하고 평가하는 데 사용되는 대규모 자연어 데이터임.
토큰: 개별 단어, 문장 부호 등 말뭉치보다 더 작은 단위의 텍스트
 - 입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'
 - 결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', 'ㄹ', '수', '있', '다', '.']

토큰화 예시

- 토큰화를 위해 토큰라이저(tokenizer) 를 사용함
 - **Tokenizer**: 텍스트 문자열을 토큰으로 나누는 알고리즘, 소프트웨어
- 토큰라이저 구축 방법
 - **공백 분할**: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화
 - **정규표현식 적용**: 정규 표현식으로 특정 패턴을 식별해 텍스트 분할
 - **어휘 사전(vocabulary)** 적용: 사전에 정의된 단어 집합을 토큰으로 사용
 - **OOV(Out of Vocab)** 문제 고려해야 함: 직접 어휘 사전을 구축하기 때문에 사전에 없는 단어나 토큰이 존재할 수 있음
 - 그렇다고 큰 어휘 사전을 구축하면?
 - 학습 비용이 커짐
 - **차원의 저주(curse of dimensionality)** 에 빠질 수 있음

: 학습 데이터의 차원이 커질수록 학습에 필요한 데이터가 증가하거나 모델의 성능이 저하되는 현상

 - 모든 토큰이나 단어를 벡터화하면 어휘 사전에 등장하는 토큰 개수만큼의 차원이 필요하고, 벡터값이 거의 모두 0이 되는 희소(sparse) 데이터로 표현됨
 - 희소 데이터의 표현 방법은 어휘 사전에 등장하는 출현 빈도만 고려해서 문장 속 토큰들의 순서나 구조 관계를 잘 표현하지 못함
- ▼ **차원의 저주 추가 설명**
 - 데이터의 차원 == 피쳐 개수
 - 벡터화된 단어.. 벡터는 어휘 사전에 있는 단어만큼의 차원을 가짐 (ex. 10,000차원 벡터)
 - 어휘 사전이 커짐 → 벡터 차원이 커짐 → 데이터가 희소해짐

: 나와 동일한 단어만 1로 표시되니까 다른 단어들이 피쳐로 많아지면 내 row가 갖는 0 값이 많아지는 것임 (ex. [0, 0, 1, 0, 0, 0, 0, 0, ...,]

→ 유의미한 패턴을 찾기 어려워짐
 - 차원이 커지면 각 차원을 커버하기 위해 훨씬 많은 데이터가 필요함

(동일한 좌표 범위에 대해 1차원 공간에선 N개의 점, 2차

원에선 N^2 개의 점, 3차원에선 N^3 개의 점으로 매꿔야 됨)

- 머신러닝 활용: 데이터세트를 기반으로 토큰화하는 방법을 학습한 머신러닝 적용

단어 및 글자 토큰화

더 정교한 토큰화 기법이 연구되고 있지만 아직 단어 토큰화, 글자 토큰화가 강력한 토큰화 기법으로 사용되고 있음

단어 토큰화 (Word Tokenization)

텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업

- 띄어쓰기, 문장 부호, 대소문자 등 특정 구분자를 활용해 토큰화 수행
- 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용됨
- string 데이터 형태는 split으로 쉽게 토큰화 가능

예제 5.1 단어 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = review.split()  
print(tokenized)
```

출력 결과

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']
```

👉 구분자만 보고 토큰화하기 때문에 '최고!', '최고!!'가 서로 다른 의미 단위로 나뉘는 등의 오류가 있음

→ 한국어 접사, 문장 부호, 오타, 띄어쓰기 오류 등에 취약함

글자 토큰화 (Character Tokenization)

텍스트 데이터를 글자 단위로 분리하는 작업

- 비교적 작은 단어 사전을 구축할 수 있음 (글자 조합인 단어보다 경우의 수가 적음)
→ 컴퓨팅 자원 절약, 전체 말뭉치를 학습할 때 같은 글자를 더 자주 학습할 수 있음
- 다음 문자를 예측하는 언어 모델링 등 시퀀스 예측 작업에 활용

예제 5.2 글자 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = list(review)  
print(tokenized)
```

출력 결과

```
['현', '실', '과', ' ', '구', '분', ' ', ' ', '불', '가', '능', '한', ' ', ' ', 'c', 'g', '.', ' ', ' ', '시',  
'각', '적', ' ', ' ', '즐', '거', '움', '은', ' ', ' ', '최', '고', '!', ' ', ' ', '더', '불', '어', ' ', ' ', 'o', 's',  
't', '는', ' ', ' ', '더', '더', '욱', ' ', ' ', '최', '고', '!', '!']
```

- string → list 형태로 변환하면 쉽게 수행할 수 있음
- 공백도 토큰으로 나뉨
- 영어는 글자 토큰화를 하면 알파벳으로 나뉘는데, 한글은 자모음이 있으므로 자모(jamo) 라이브러리 등을 이용해 자소 단위 토큰화를 해야 함
 - 자모 변환 함수: 입력된 한글 문자열을 유니코드 U+1100~U+11FE 사이의 조합형 한글 자모로 변환
 - 한글 인코딩 방식 - 조합형: 글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글을 표현하는 방식 / 완성형: 조합된 글자 자체에 값을 부여해 인코딩하는 방식
 - `jamo.h2j` : hangeul to jamo - 완성형으로 입력된 한글을 조합형 한글로 변환하는 함수

자모 변환 함수

```
retval = jamo.h2j(  
    hangul_string  
)
```

- `jamo.j2hcj` : jamo to hangeul? - 조합형 한글 문자열을 자소 단위로 나눠 반환하는 함수

한글 호환성 자모 변환 함수

```
retval = jamo.j2hcj(  
    jamo  
)
```

👍 비교적 적은 크기의 단어 사전이 구축되어 OOV 감소 / 접사, 문장 부호의 의미 학습 가능

👉 개별 토큰에 아무런 의미가 없어 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야 하는데, 토큰 조합 방식을 사용해 문장 생성이나 개체명 인식(Name Entity Recognition) 등을 구현할 경우 다의어나 동음이의어가 많은 도메인에서 구별하는 것이 어려울 수 있음.

- 개체명 인식 (Name Entity Recognition): 자연어 내에서 사전에 정의된 Named Entity를 인식하고 추출하는 자연어 처리의 한 분야

모델 입력 시퀀스(sequence)의 길이가 길어질수록 연산량이 증가함

- 시퀀스: 순서가 의미를 가지는 데이터. 자연어 데이터에서는 거의 모든 데이터가 의미 있는 순서를 지니고 있음

형태소 토큰화 (Morpheme Tokenization)

텍스트를 형태소 단위로 나누는 토큰화 방법

언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업

- 한국어 등 각 단어가 띄어쓰기로 구분되지 않는 교착어(Agglutinative Language)인 언어에서 중요하게 수행됨 (↔ 영어)

형태소 어휘 사전 (Morpheme Vocabulary)

어휘 사전 중 각 단어의 형태소 정보를 포함하는 사전

- ex. "그녀는 그에게 인사를 했다" & "그는 그녀에게 인사를 했다"
→ ['-는', '-에게', '그', '나', '그녀'] 식으로 토큰화 가능
- 일반적으로 각 형태소가 어떤 품사에 속하는지, 해당 품사의 뜻 등 정보도 함께 제공
→ 품사 태깅(POS Tagging): 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사를 태깅하는 작업 - 으로 문맥을 고려할 수 있어 더 정확한 분석 가능

KoNLPy

한국어 자연어 처리를 위해 개발된 라이브러리로 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공

- 자바 기반 한국어 형태소 분석기 (jdk 기반으로 개발되었기 때문에 jdk가 설치되어 있어야 사용 가능)
- Okt(Open Korean Text, sns 텍스트 데이터 기반), 꼬꼬마(Kkma, 국립국어원에서 배포한 세종 말뭉치 기반), 코모란(Komoran), 한나눔(Hannanum), 메캅(Mecab) 등의 형태소 분석기 지원
(메캅은 맥, 우분투에서만 지원)

Okt

- `okt.nouns` 명사 추출
- `okt.phrases` 구 추출
- `okt.morphs` 형태소 추출
- `okt.pos` 품사 태깅

Kkma

- `kkma.nouns` 명사 추출
- `kkma.sentences` 문장 추출
- `kkma.morphs` 형태소 추출
- `kkma.pos` 품사 태깅

(실습)

NLTK (Natural Language Toolkit)

토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등의 기능 제공

- 주로 영어 자연어 처리를 위해 개발됐지만 다른 다양한 언어의 자연어 처리를 위한 데이터와 모델도 제공함

punkt, punkt_tag 기반

- `nltk.tokenize.word_tokenize` 단어 토큰라이저: 공백 기반 분리
- `nltk.tokenize.sent_tokenize` 문장 토큰라이저: 구두점 기반 분리

averaged_perceptron_tagger, averaged_perceptron_tagger_eng 기반

→ 35개 품사 태깅 가능

- `nltk.tag.pos_tag` 품사 태깅

(실습)

spaCy

사이썬(Cython) 기반 오픈소스 라이브러리로, 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반 자연어 처리 라이브러리

- nltk 모델보다 더 크고 복잡하며 더 많은 리소스 요구
- gpu 가속 제공
- 24개 이상 언어로 사전 학습된 모델 제공

<https://spacy.io/usage> 에서 파이프라인 설치할 수 있음

하위 단어 토큰화 (Subword Tokenization)

? 형태소 분석기는 모르는 단어를 적절한 단위로 나누는 것에 취약하기 때문에 더이상 사용되지 않는 단어, 신조어, 고유어, 외래어, 전문용어 등을 잘 처리하지 못함
→ 어휘 사전의 크기를 키워야 할 가능성 ⇒ OOV 대응하기 어려워짐

하위 단어 토큰화

- 하나의 단어가 빈번하게 사용되는 하위 단어 Subword의 조합으로 나누어 토큰화하는 방법
- ex. Reinforcement → Rein, force, ment 로 나눠 처리
- 단어의 길이를 줄이므로 처리 속도가 빠름
- OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제 완화
- 방법: 바이트 페어 인코딩, 워드피스, 바이트 수준 바이트 페어 인코딩(BBPE), 유니그램 모델 등
 - BBPE: 유니코드 단위가 아닌 바이트 단위에서 토큰화
 - 유니그램(Unigram): 크기가 큰 어휘 사전에서 덜 필요한 토큰을 제거하며 학습

바이트 페어 인코딩 (Byte Pair Encoding, BPE)

= 다이그램 코딩(Digram Coding)

텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합을 찾아 부호화하는 압축 알고리즘 (글자 빈도수 측정 → 가장 빈도수가 높은 글자 쌍 탐색)

- 연속된 글자 쌍이 더이상 나타나지 않거나, 정해진 어휘 사전 크기에 도달할 때까지 조합 탐색과 부호화를 반복
 - 원문: abracadabra
 - Step #1: AracadAra
 - Step #2: ABcadAB
 - Step #3: CcadC
- 자주 등장하는 단어 → 하나의 토큰으로 토큰화
- 덜 등장하는 단어 → 여러 토큰의 조합으로 표현됨
- 자주 등장하는 글자 쌍을 찾으면 어휘 사전에 추가함

- ex)
 - 말뭉치에서 각 **단어**가 등장한 빈도를 계산해 만든 빈도 사전과 어휘 사전

- 빈도 사전: ('low', 5), ('lower', 2), ('newest', 6), ('widest', 3)
- 어휘 사전: ['low', 'lower', 'newest', 'widest']

- 빈도 사전을 바이트 페어 인코딩으로 재구성

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

- 가장 자주 등장한 글자 쌍은 'es'이므로 어휘 사전에 추가

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'es', 't', 6), ('w', 'i', 'd', 'es', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es']

- 빈도 사전에서 빈도수가 높은 'es', 't' 쌍도 'est'로 병합하고 어휘 사전에 추가

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est']

'new e r', 'wid e r', 'low est' 등의 토큰화가 가능함

센텐스피스(Sentencepiece)

- 구글에서 개발한 오픈소스 하위 단어 토큰라이저 라이브러리
- 바이트 페어 인코딩과 유사한 알고리즘으로 입력 데이터를 토큰화하고 단어 사전 생성

표 5.5 SentencePieceTrainer 매개변수

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토큰라이저가 다룰 수 있는 글자의 비율
model_type	토큰라이저 알고리즘 ■ unigram ■ bpe ■ char ■ word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

코포라(Korpora)

- 국립국어원이나 AI Hub에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리
- 파이썬 API 제공

(실습)

워드피스(Wordpiece)

새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 **확률**을 계산해 가장 높은 확률을 가진 하위 단어 선택해서 병합

수식 5.1 글자 쌍 병합 점수 수식

$$score = \frac{f(x,y)}{f(x)f(y)}$$

- f: 빈도 함수
- x, y: 병합하려는 하위 단어
- f(x, y): xy 글자 쌍의 빈도

- ex)

- 'es' 9번 등장, 'e' 17번, 's' 9번 등장 → score가 빈도에 비해 높지 않음
 - 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']
- 'i', 'd' 3번 등장, 'id' 3번 등장 → score 높음 ⇒ 병합
 - 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
 - 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id']

토큰나이저스 (Tokenizers) 라이브러리를 이용한 실습

- 정규화(Normalization): 텍스트 형식 표준화, 일부 문자를 대체하거나 제거해 모호한 경우를 방지 (불필요 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등)

표 5.6 정규화 클래스

이름	설명
NFD()	NFD 유니코드 정규화
NFKD()	NFKD 유니코드 정규화
NFC()	NFC 유니코드 정규화
NFKC()	NFKC 유니코드 정규화
Lowercase()	입력 문장의 모든 대문자를 소문자로 변환
Strip()	입력 문장의 양 끝에 있는 공백 제거
StripAccents()	모든 엑센트 기호 제거
Replace()	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
BertNormalizer()	BERT 모델에 사용된 정규화 적용
Sequence()	여러 개의 정규화 모듈을 병합해 순서대로 수행

- 사전 토큰화(Pre-tokenization): 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나누는 기능

표 5.7 사전 토큰화 클래스

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
CharDelimiterSplit()	문자 기준으로 문자열을 나눈다.
Digits()	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
Punctuation()	구두점을 기준으로 입력 문자열을 나눈다.
Metaspace(replacement="_")	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다. ▪ 기본값: _(U+2581)
Whitespace()	단어 경계(공백과 구두점)를 기준으로 사전 토큰화를 진행한다. ▪ <code>\w+![^\\w\\s]+</code> 정규 표현식을 적용한다.
WhitespaceSplit()	공백 문자를 기준으로 입력 문자열을 나눈다.
Split(pattern, behavior)	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. ▪ behavior - removed: 삭제 - isolated: 개별 토큰 처리 - merged_with_previous: 이전 문자열과 결합해 토큰 처리 - merged_with_next: 다음 문자열과 결합해 토큰 처리 - contiguous: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리

(실습)

6장_임베딩

토큰화: 문장 → 여러 개의 토큰 (텍스트)

텍스트 벡터화(Text Vectorization): 텍스트 → 숫자

- 원-핫 인코딩(One-Hot Encoding): 문서에 등장하는 각 단어를 고유한 id로 매핑한 뒤, 해당 인덱스 값을 1로 설정하고 나머지는 모두 0으로 표시

표 6.1 원-핫 인코딩 매핑 테이블

색인	0	1	2	3
토큰	I	like	apples	bananas

- i like apples: [1,1,1,0] / i like bananas: [1,1,0,1]
- 빈도 벡터화: 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식

→ 원-핫 인코딩, 빈도 벡터화 모두 쉽고 간단하지만;

- 벡터의 희소성(Sparsity)이 큼
- 만들어지는 벡터 차원에 비해 하나의 입력 문장 내에 존재하는 토큰의 수는 그에 비해 적음
→ 컴퓨팅 비용 증가(가성비x), 차원의 저주
- 텍스트의 벡터는 입력 텍스트의 의미를 내포하지 x
- 워드 임베딩(Word Embedding)
 - 단어 → 고정된 길이의 실수 벡터로 표현
 - 단어의 의미 → 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계 추론
 - 동적 임베딩(Dynamic Embedding): 고정된 임베딩을 학습하면 다의어, 문맥 정보를 다루기 어려우므로 인공신경망을 활용해 임베딩하는 것

언어 모델 (Language Model)

입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델

- 자동 번역, 음성 인식, 텍스트 요약 등의 자연어 처리 분야에서 활용
- ex. input: "안녕하세요", $P(\text{"만나서 반갑습니다"})$, $P(\text{"아니오"})$ 등의 확률 계산 → 불가능
- 문장 전체에 대한 예측 대신 하나의 토큰 단위로 예측

자기회귀 언어 모델 (Autoregressive Language Model)

입력된 문장들의 조건부 확률을 이용해 다음에 올 단어 예측

- 언어 모델에서의 조건부 확률: 이전 단어들의 시퀀스가 주어졌을 때 다음 단어에 대한 확률
= (다음 단어 포함 시퀀스 확률) / (이전 단어까지의 시퀀스 확률)

수식 6.1 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

- 조건부 확률의 연쇄법칙: 다음에 등장하는 단어의 확률을 이전 단어들의 조건부 확률을 이용해 계산

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = P(w_1)P(w_2 | w_1) \dots P(w_t | w_1, w_2, \dots, w_{t-1})$$

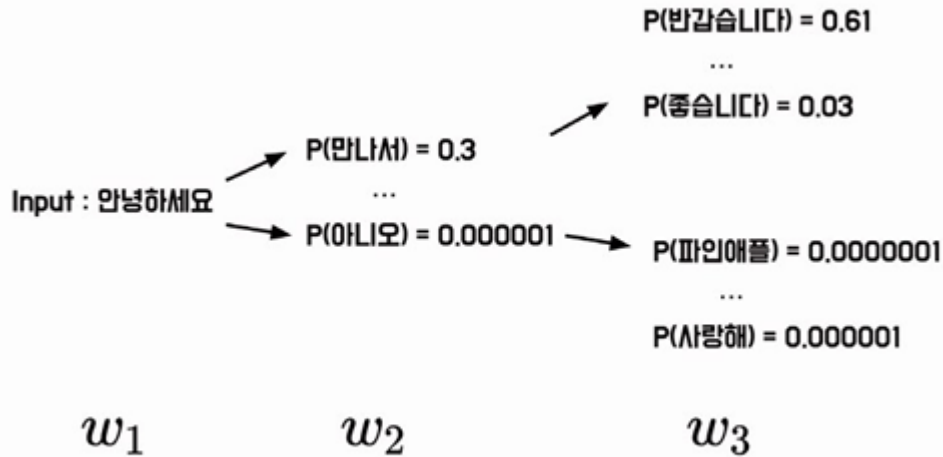


그림 6.2 조건부 확률의 연쇄법칙을 통한 자기회귀 언어 모델 구조

- “자기회귀”: 모델의 출력값이 모델의 입력값으로 사용됨

통계적 언어 모델 (Statistical Language Model)

시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률 예측

- 마르코프 체인(Markov Chain)로 구현됨: 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태 예측 - 빈도 기반의 조건부 확률 모델
 - 빈도 기반의 조건부 확률 모델: 주어진 데이터에서 각 변수가 발생한 빈도수를 기반으로 확률 계산

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

→ 안녕하세요 전체 중 '만나서'와 쓰인 확률, '반갑습니다'와 쓰인 확률 계산

- 단어의 순서, 빈도에만 기초에 문장의 확률 예측 → 문맥을 제대로 파악하지 못하면 부적절한 결과
- 데이터 희소성(Data sparsity): 관측한 적이 없는 데이터를 예측하지 못함
- 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를 생성
→ 새로운 문장 생성, 다양한 종류의 텍스트 데이터 학습 가능
- 대규모 자연어 데이터 처리 효과적
- 최근 자연어 처리 기법: 언어 모델을 활용해 가중치 사전 학습
 - GPT(Generative Pre-trained Transformer), BERT(Bidirectional Encoder Representations from Transformers)

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

이렇게 문장을 생성해서 모델을 사전학습 시킨다~

N-gram

텍스트에서 **N개의 연속된 단어 시퀀스** 를 하나의 단위로 취급 → 특정 단어 시퀀스가 등장할 확률 추정

- N=1: 유니그램 / N=2: 바이그램 / N=3: 트라이그램 / N≥4: N-gram

Unigram : 안녕하세요 만나서 진심으로 반가워요 안녕하세요. 만나서. 진심으로. 반가워요

Bigram : 안녕하세요 만나서 진심으로 반가워요 안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요

Trigram : 안녕하세요 만나서 진심으로 반가워요 안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

그림 6.3 N이 1, 2, 3인 N-gram

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t \mid w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

- 단어의 순서가 중요한 자연어 처리 작업 및 문자열 패턴 분석에 활용

TF-IDF (Term Frequency-Inverse Document Frequency)

= 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적 가중치

- BoW(Bag-of-Words)에 가중치를 부여하는 방법
- BoW 벡터: 문서, 문장 → 단어의 집합으로 표현해 각 단어가 등장한 빈도를 기록한 벡터

표 6.2 BoW 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

- 모든 단어가 동일한 가중치를 가짐

단어 빈도(Term Frequency, TF)

= 문서 내에서 특정 단어의 빈도수

수식 6.5 단어 빈도

$$TF(t, d) = count(t, d)$$

- TF  → 자주 사용되는 단어, 전문 용어 혹은 관용어

문서 빈도 (Document Frequency, DF)

= 단어가 몇 개의 문서에서 등장하는가

수식 6.6 문서 빈도

$$DF(t, D) = count(t \in d: d \in D)$$

- DF  → 특정 단어가 많은 문서에서 등장하므로 일반적인 단어, 중요도 낮을 수 있음
- DF  → 특정 단어가 적은 수의 문서에만 등장하므로 그 문서만의 특수한 단어, 중요도 높을 수 있음

역문서 빈도 (Inverse Document Frequency, IDF)

= log(전체 문서 수 / 문서 빈도)

→ 문서 내에서 특정 단어가 얼마나 중요한가

수식 6.7 역문서 빈도

$$IDF(t, D) = \log\left(\frac{count(D)}{1 + DF(t, D)}\right)$$

- 로그 취하는 이유: 10,000개 문서, 특정 단어 빈도 1 → IDF = 5,000 (너무 큰 값)
⇒ 정교한 가중치를 얻고자 하는 것

TF-IDF

= (문서 빈도) x (역문서 빈도)

수식 6.8 TF-IDF

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, d)$$

- 하나의 문서 내에서만 자주 등장하는 단어 → TF-IDF 
- 여러 문서들에서 자주 등장하는 단어 = 관사, 관용어 등 → TF-IDF 

(사이킷런 실습)